Build Blueprint — SoundCloud + Social Network (MVP)

Product Direction

Product Type: SoundCloud + Social Network
Model: UGC + Verified Artists
Strategy: Mobile-first + Social-first
Goal: MVP launch in 2–4 months without painful rewrites later.

Core Product Vision

A social music platform where:

- users upload tracks
- creators grow audiences
- verified artists publish music
- users follow, like, repost, comment
- playlists are social
- discovery drives engagement

Positioning:

> SoundCloud × Instagram engagement layer

---

1. Final Tech Stack

Web

- Next.js (App Router)
- TailwindCSS
- TanStack Query
- Zustand

Mobile

- Flutter
- Riverpod
- go_router
- Dio

Backend

- NestJS (TypeScript)
- REST API + WebSocket

Database

- PostgreSQL

Cache / Queue

- Redis
- BullMQ

Storage

- Cloudflare R2

Audio Processing

- FFmpeg

Search

- Meilisearch

Realtime

- Socket.IO

Monitoring

- Sentry

Notifications

- Firebase Cloud Messaging (FCM)

---

2. MVP Features (STRICT)

Only build what is necessary for launch.

Authentication

- Email/password
- Google login
- Apple login (iOS)
- JWT auth
- refresh token

User Profile

- username
- avatar
- bio
- followers/following
- creator badge
- verified artist badge

Music Upload

- upload track
- cover image
- title
- description
- genre
- tags
- visibility (public/private)

Music Player

- play/pause
- seek
- queue
- background playback
- lockscreen support

Social

- follow users
- likes
- comments
- repost
- notifications

Playlist

- create playlist
- add/remove tracks
- public/private

Feed

- following feed
- trending feed
- newest feed

Search

- user search

- track search
- playlist search

Artist Verification

- request verification
- admin approval

---

3. DO NOT BUILD IN MVP

Avoid these until users exist:

- recommendation AI
- advanced ranking system
- subscription/premium
- copyright matching system
- live audio
- chat system
- collaborative playlists
- advanced analytics

These are V2/V3.

---

4. Architecture Philosophy

Use:

Modular Monolith

NOT microservices.

Why?

- faster shipping
- easier debugging
- cheaper infra
- less DevOps pain

Structure it so modules can later be extracted.

---

5. Monorepo Structure

```
apps/
  api/
```

```
  web/
  mobile/

packages/
  shared-types/
  ui/
  constants/
```

apps/api

NestJS backend

apps/web

Next.js

apps/mobile

Flutter app

shared-types

shared DTOs/types

---

6. Backend Architecture (NestJS)

Modules:

```
auth
users
profiles
tracks
uploads
streaming
playlists
comments
likes
followers
notifications
search
feed
admin
moderation
analytics
```

Each module contains:

```
controller
service
repository
dto
entity
```

---

7. Database Design

Core tables:

users

- id
- username
- email
- password_hash
- role
- verified_status
- avatar_url
- bio
- created_at

tracks

- id
- user_id
- title
- description
- cover_url
- audio_url
- duration
- waveform_json
- genre
- visibility
- play_count
- created_at

playlists

- id
- user_id
- title
- cover_url
- visibility

playlist_tracks

- playlist_id
- track_id
- order_index

followers

- follower_id
- following_id

likes

- user_id
- track_id

comments

- id
- user_id
- track_id
- content
- created_at

reposts

- user_id
- track_id

notifications

- id
- user_id
- type
- actor_id
- entity_id
- is_read

---

8. Upload Pipeline

CRITICAL PART.

Flow:

1. user uploads audio
2. upload stored in R2
3. job added to BullMQ
4. FFmpeg processes file

5. optimized versions generated
6. waveform generated
7. metadata saved
8. track becomes published

Why queue?

Because audio processing blocks server threads.

---

9. Audio Processing

FFmpeg jobs:

- normalize audio
- bitrate optimization
- waveform generation
- preview clip generation

Store:

```
original/
processed/
waveforms/
previews/
```

---

10. Feed Strategy (MVP)

Keep it simple.

Following Feed

Tracks uploaded by followed users.

Trending Feed

Based on:

- likes
- comments
- reposts
- play velocity

New Feed

Recent uploads.

Do NOT build ML recommendation yet.

---

11. Search Strategy

Use Meilisearch.

Indexes:

- users
- tracks
- playlists

Search fields:

- title
- username
- tags
- genre

---

12. Flutter Architecture

Feature-first architecture.

```
features/
  auth/
  profile/
  player/
  tracks/
  playlists/
  feed/
  comments/
  notifications/
```

State Management:

Riverpod.

Routing:

go_router.

Networking:

Dio.

---

13. Next.js Architecture

Pages:

- Home
- Explore
- Artist Profile
- Playlist
- Track Page
- Upload Page
- Settings

SEO:

Track pages must be indexable.

Example:

```
/user/artist-name
/track/song-name
/playlist/my-playlist
```

---

14. Authentication Strategy

JWT Access Token

+ Refresh Token

Storage:

Web: httpOnly cookies

Mobile: secure storage

---

15. Notifications

MVP Notifications:

- follow
- comment
- like
- repost

Realtime: Socket.IO

Push: FCM

---

16. Security

Required:

- rate limiting
- upload limits
- MIME validation
- profanity moderation
- bot prevention
- IP throttling

Never trust client-side validation.

---

17. Deployment

Web

Vercel

Backend

Railway or VPS

Database

Managed PostgreSQL

Redis

Upstash

Storage

Cloudflare R2

Keep infra simple in MVP.

---

18. Weekly Roadmap (12 Weeks)

Week 1–2

Project setup Auth Database Monorepo

Week 3–4

Profiles Follow system Track upload Player

Week 5–6

Playlist Likes Comments

Week 7–8

Feed Search Realtime notifications

Week 9–10

Flutter polish Web polish Performance

Week 11–12

Testing Analytics Deployment Beta launch

---

19. AI-Assisted Workflow

Never ask AI:

> build entire app

Instead:

Ask module-by-module.

Example order:

1. Auth module
2. User module
3. Upload system
4. Track player
5. Playlist
6. Feed
7. Notifications

Always demand:

- architecture first
- tests
- DTOs
- folder structure
- production-ready code

---

20. Biggest Mistakes To Avoid

1. Overbuilding recommendation system early
2. Firebase-only backend
3. Microservices too early
4. Bad upload pipeline
5. Ignoring analytics
6. Weak search UX
7. Slow music player UX
8. Building too many features before launch

Rule:

> Launch ugly, but stable. Improve with real user behavior.

End of Phase 1 Blueprint.